

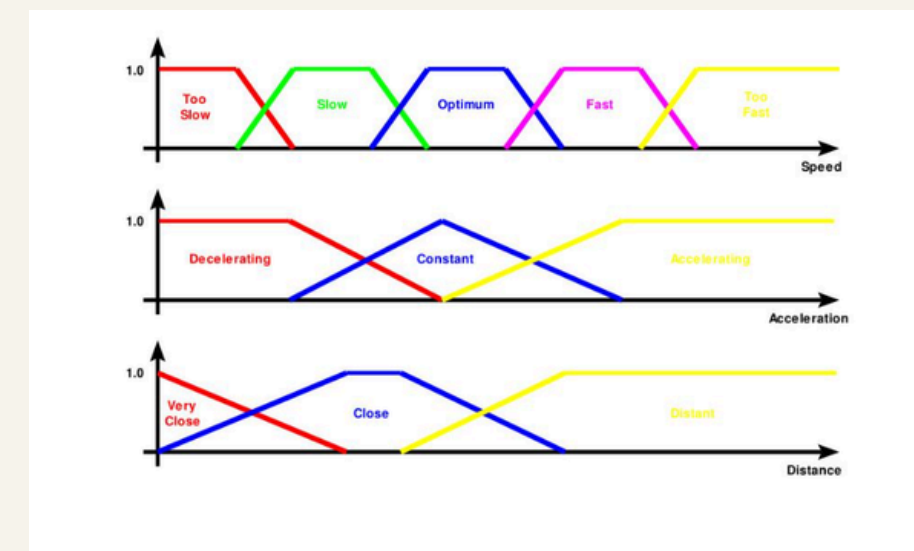
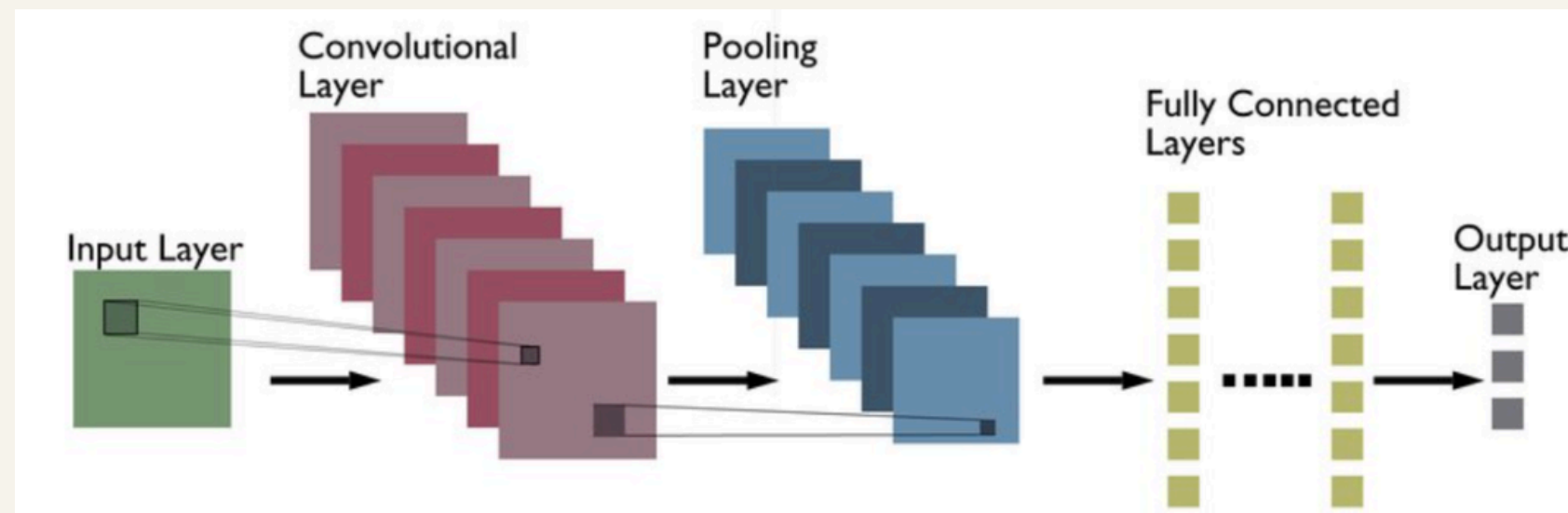
# CSE-458 (Soft Computing) Project

Submitted by:

- Tanmay Tyagi - 22095138

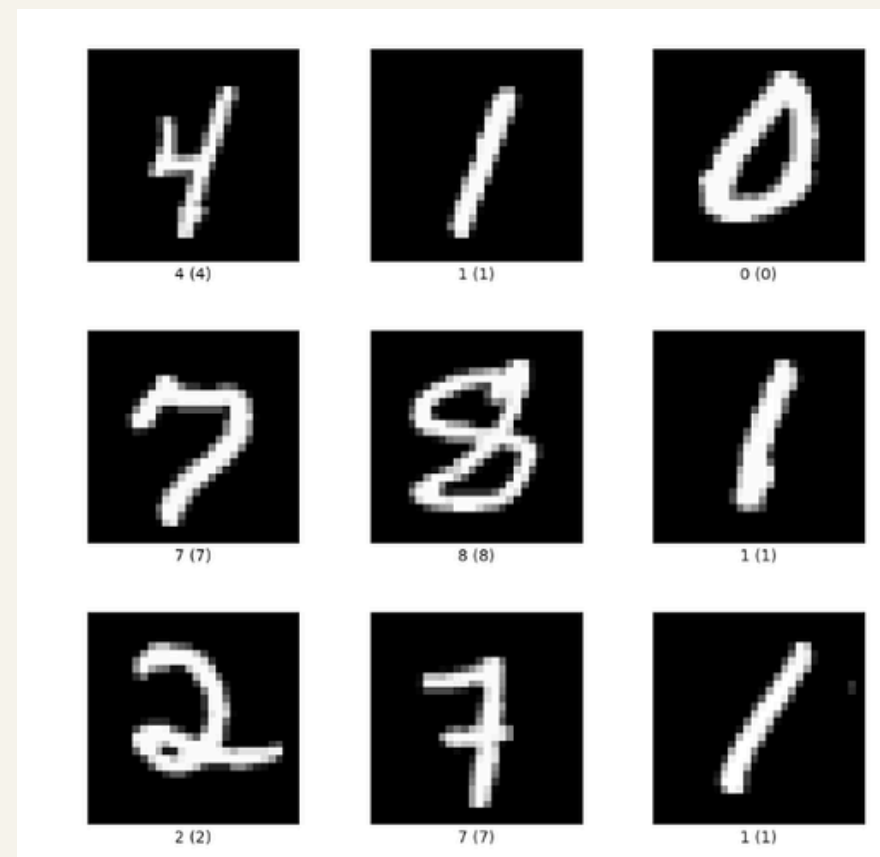
# OBJECTIVE

To combine fuzzy logic with CNN for image classification on datasets like MNIST with improved interpretability.



# DATASET

**The MNIST (Modified National Institute of Standards and Technology) dataset is a classic benchmark for image classification tasks. It has simple and clean digit images and is widely used for prototyping deep learning models.**



# FUZZY LOGIC OVERVIEW

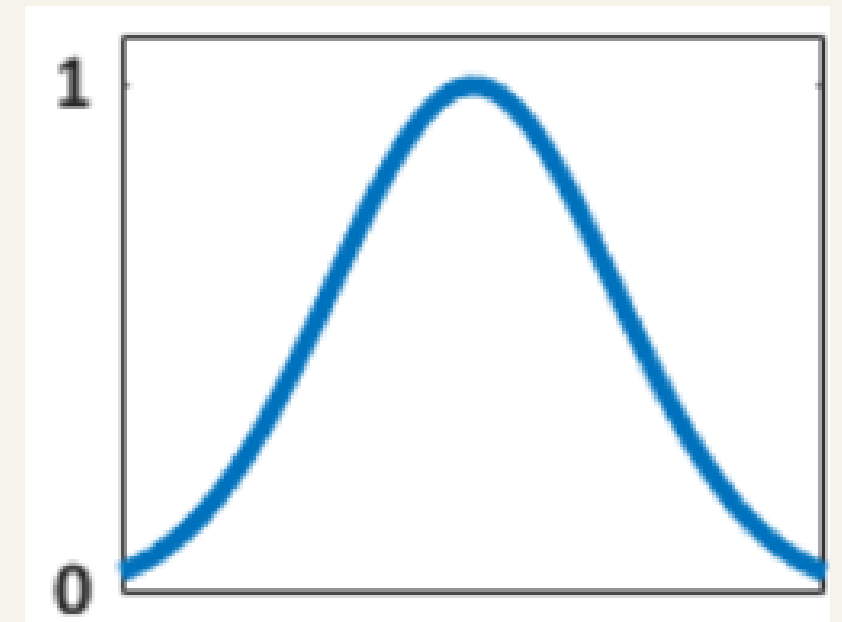
**Fuzzy logic is the logic underlying approximate, rather than exact, modes of reasoning. It allows degrees of truth, ranging from 0 to 1.**

**We apply a Gaussian Membership Function to the input pixel values to transform them into fuzzy values before feeding them to the convolutional layers.**

# GAUSSIAN MEMBERSHIP FUNCTION

A Gaussian Membership Function (GMF) is a smooth, bell-shaped curve used to determine how strongly an input belongs to a fuzzy set.

In our model, each pixel value is passed through a GMF before convolution, converting crisp values into fuzzy activations. This enhances feature extraction under uncertain or noisy conditions.



```
def gaussian_mt(x, mean, sigma):  
    return tf.exp(-tf.square(x - mean) / (2 * tf.square(sigma)))
```



# FUZZY CONV. LAYER

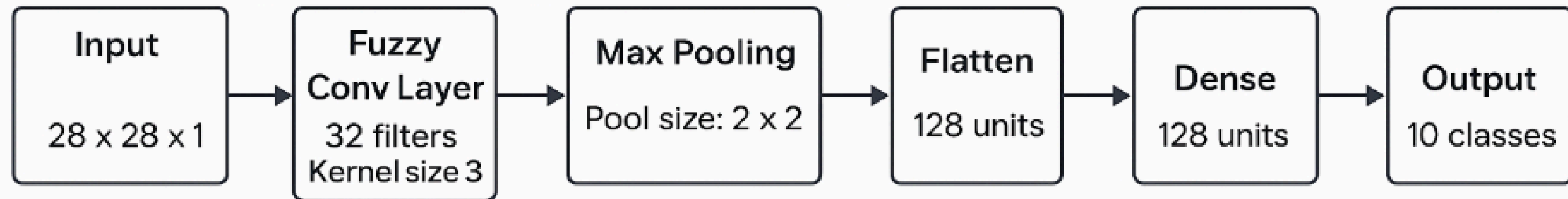
```
# Fuzzy Convolutional Layer
class FuzzyConvLayer(Layer):
    def __init__(self, filters, kernel_size):
        super(FuzzyConvLayer, self).__init__()
        self.filters = filters
        self.kernel_size = kernel_size
        self.kernel = None
        self.mean = None
        self.sigma = None

    def build(self, input_shape):
        in_channels = input_shape[-1]
        self.kernel = self.add_weight(shape=(self.kernel_size, self.kernel_size, in_channels, self.filters),
                                      initializer="random_normal",
                                      trainable=True,
                                      dtype=tf.float32)
        self.mean = self.add_weight(shape=(1,), initializer="zeros", trainable=True, dtype=tf.float32)
        self.sigma = self.add_weight(shape=(1,), initializer="ones", trainable=True, dtype=tf.float32)

    def call(self, inputs):
        inputs = tf.cast(inputs, tf.float32)
        fuzzy_inputs = gaussian_mf(inputs, self.mean, self.sigma)
        fuzzy_output = tf.nn.conv2d(fuzzy_inputs, self.kernel, strides=[1, 1, 1, 1], padding='SAME')
        return fuzzy_output
```

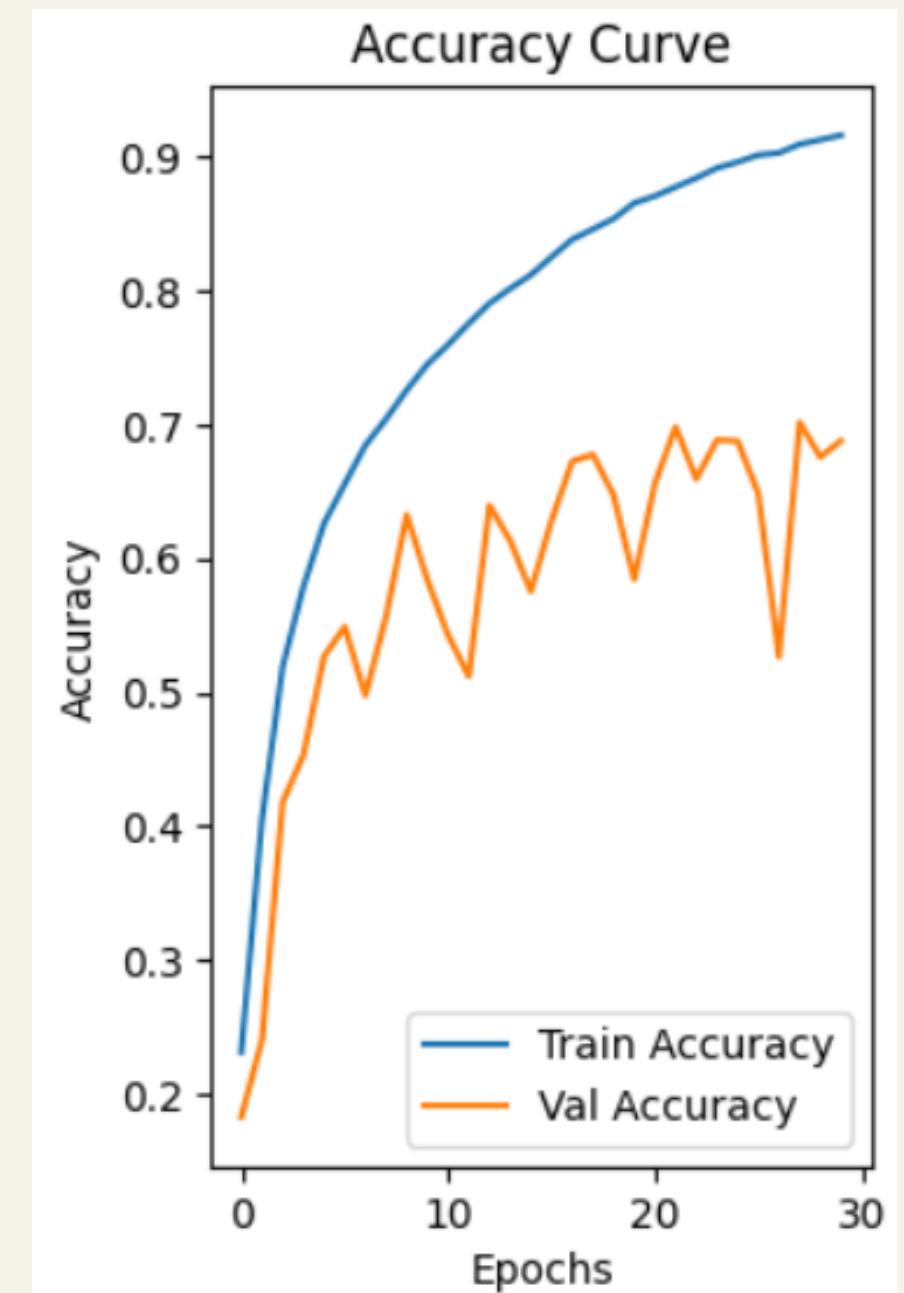
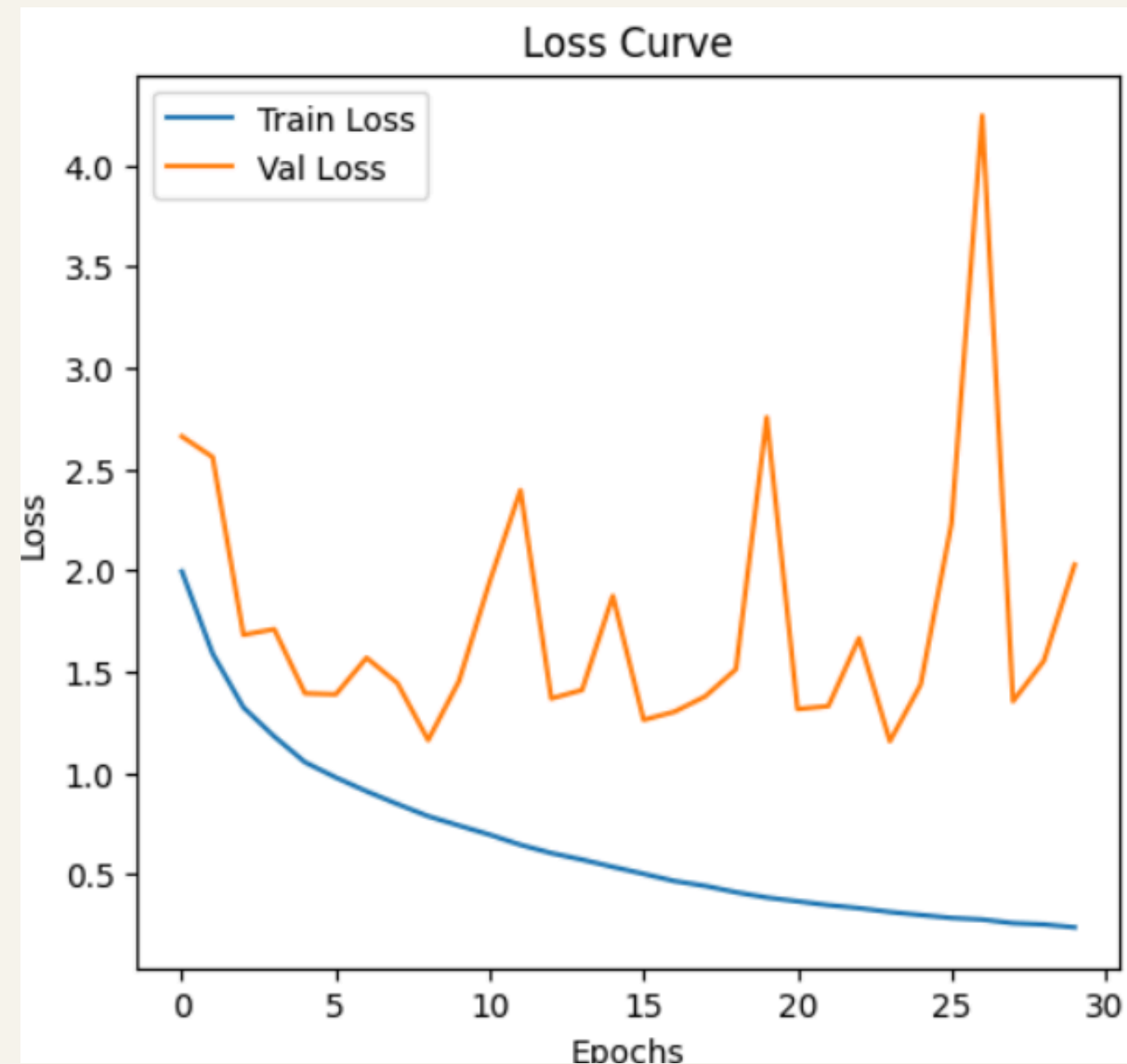
- It is a custom layer that first applies Gaussian MF
- Then it performs convolution
- It learns fuzzy + CNN parameters together

# MODEL ARCHITECTURE



# TRAINING AND EVALUATION

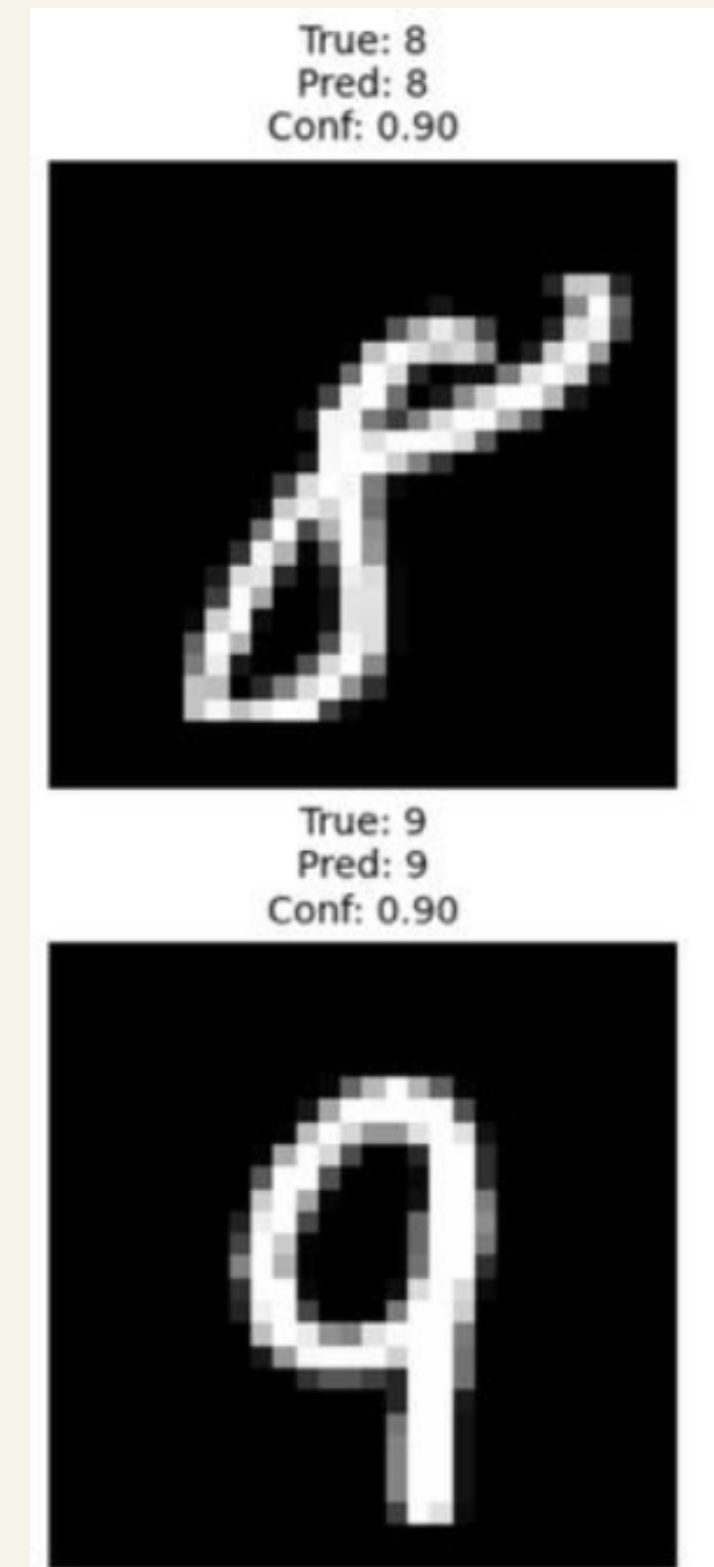
- **Optimizer:**  
Adam
- **Loss:**  
Categorical  
Crossentropy
- **Epochs:** 10
- **batch\_size=64**





# RESULTS AND OBSERVATIONS

- **Test Accuracy: ~97.32%**
- **Model generalizes well**
- **Fuzzy layers reduce sensitivity to noise**
- **Training is stable**



The background features three vertical stripes on the left: a wide pink stripe, a narrower blue stripe, and a medium-width beige stripe. The right side of the image is a light cream color, decorated with two rectangular areas of a pink dot pattern. The top area is a 10x10 grid of dots, and the bottom area is a 10x8 grid of dots.

**THANK YOU**